

Diversifying Search Results for Homonym Queries via Transformer Architectures

Emmanuel Diehl

TU Dresden

emmanuel.diehl@mailbox.tu-dresden.de

Manuel Berger

TU Dresden

manuel.berger@mailbox.tu-dresden.de

Franz Martin Schmidt

TU Dresden

franz_martin.schmidt@mailbox.tu-dresden.de

Maria Hampel

TU Dresden

maria.hampel@mailbox.tu-dresden.de

Reiner Frank Stolle

TU Dresden

reiner_frank.stolle@mailbox.tu-dresden.de

Abstract

This document is part of the "Information Retrieval Praktikum" course and details the creation of a search engine that retrieves diverse results for homonym queries. Wikipedia pages and the Wikidata database were selected as the underlying dataset to evaluate the diversity of returned results.

1 Introduction

Existing Information Retrieval (IR) methods such as tf-idf or ColBERT (Khattab and Zaharia, 2020) are able to rate the relevance of a document in regards to a given query. When using homonyms (single words with multiple semantic meanings) as search terms, this poses the challenge of results potentially only featuring results for a given meaning, that might not match the users need for information. Imagine a search for the speed of a jaguar. Depending on whether the user is interested in the make of cars or the species of cat, their information need might not be satisfied by a list of results featuring only one or the other.

As part of the "Information Retrieval Praktikum" course at TU Dresden, a search engine with a focus on returning a diverse set of results for a homonym query was created¹.

¹https://github.com/emmdie/IR_Praktikum

1.1 Research Question and Goals

The goal of this project was to develop a search engine that could provide diverse results for homonym queries. Given the open nature of the task, the following objectives were established: using Wikipedia as the dataset, restricting searches to the first paragraph of articles, and focusing on single-word queries to avoid limiting results based on additional context. A front-end interface was also built for the search engine to enhance the user experience and enable switching between the different approaches and settings described in subsequent chapters. A brief overview of the frontend can be found in the appendix.

1.2 Outline

This report briefly describes existing approaches to the problem before detailing a static approach used for searching with precomputed semantic representations of search terms. A second dynamic approach is also described, in which the clustering of results is performed during runtime. Subsequently, the evaluation approach, including the dataset and queries used, is detailed. Finally, the limitations of this work are provided, along with a summary and outlook.

2 History of Homonyms in Information Retrieval

To contextualize the task at hand, the present chapter aims to give a brief overview of a selection of methods used to deal with the problems created by homonyms in the context of information retrieval.

2.1 Neglecting Homonyms

Failing to address the specific issues introduced by homonyms might lead to the returned documents mainly being relevant to one or a few of the homonyms semantic meanings. A search for the term "bow" might lead to the top results of a search being populated exclusively by results regarding archery, without containing any regarding the part of a ship.

2.2 Thesauri

One approach to resolving this issue is the use of Thesauri, as described by [Krovetz and Croft \(1992\)](#). A query for the speed of a jaguar might also include related words such as "car" or "cat" to return a greater number of diverse results by including related words and alternate meanings. However, even large dictionaries may not offer an exhaustive list of all a word's meanings, as a word can be used in a specialized way in a specific field, and new words are constantly being adopted. Furthermore, not all of the added terms necessarily lead to additional relevant content.

2.3 Pseudo-Relevance Feedback

Pseudo-relevance feedback (PRF) can be utilized to resolve homonymy in information retrieval. As described by [Dian Paskalis and Khodra \(2011\)](#) it can do so by leveraging the assumption that the top-ranked documents from an initial search are likely to be relevant. In the context of homonyms, PRF helps refine the query by selecting terms from these top-ranked documents that provide context. This method implicitly identifies and assumes which sense of the ambiguous term is most likely, based on co-occurrence patterns and document relevance. By iterating this process, PRF can adjust the query to reflect the appropriate meaning of the homonym, thus improving retrieval accuracy without requiring explicit user input or advanced disambiguation techniques. However, this once again falls short, if a diverse set of results is desired, as more niche meanings might not necessarily occur among the initial top-ranked documents. Pseudo-Relevance

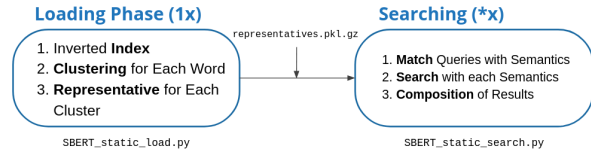


Figure 1: System architecture for static semantic retrieval.

feedback was therefore not chosen as a promising approach.

2.4 Transformers

A key aspect of the Transformer is its ability to consider terms within their surrounding context. As described by [García \(2021\)](#), this allows for the effective distinction between homonyms' different semantic meanings as long as the phrases differ sufficiently. Researches such as [Maudslay and Teufel \(2022\)](#) have been able to link different embeddings of a word to its different senses. In the context of a multi word query, accessing and comparing the embeddings therefore allows for the retrieval of documents relevant to a given word in the specific context of the query. Context can also be leveraged to classify documents by comparing the embeddings of the homonym in each document. Returning to the example from the introduction, the embeddings of the word "jaguar" would differ drastically between a car magazine and a zoo leaflet. These aspects make transformers an appealing tool for the task at hand and motivated their application in subsequent chapters.

3 Static Approach

3.1 Overview of the Static Approach

The static approach focuses on learning the semantics of words during a training phase. At query time, precomputed semantic representations are used to perform the search, eliminating the need for runtime clustering. A key motivation for this design is to ensure full semantic coverage — no meaning is missed during retrieval, provided it was captured in the loading phase. The approach leverages SBERT ([Reimers and Gurevych, 2019](#)) for sentence encoding and uses cosine similarity (i.e., dot product on normalized vectors) to rank results. Figure 1 illustrates the overall system architecture.

3.2 Loading Phase

In the first phase, an inverted index is constructed that maps each word to the set of documents in

which it occurs. This index serves as the foundation for subsequent semantic clustering and representation.

To capture polysemy, a clustering algorithm is applied to the embedding representations of all documents in the posting list for each word. The underlying assumption is that each cluster represents a distinct semantic usage of the word.

Two clustering methods were explored:

- **HDBSCAN** (McInnes et al., 2017), a density-based clustering algorithm, was chosen for its ability to automatically determine the number of clusters. Parameters were iteratively tuned to yield fewer than ten clusters per word to ensure a manageable number of semantic representations.
- **K-means** (Wu, 2012) served as a simpler, more computationally efficient baseline. However, it requires a fixed number of clusters, which limits adaptability to varying semantic complexity.

Each cluster is represented by a single vector, computed as the centroid of all embeddings in that cluster. These centroids serve as semantic proxies for the word usages and are later used during query-time retrieval.

3.3 Searching Phase

At runtime, the system first matches a query term to its precomputed semantic interpretations by retrieving the corresponding cluster representatives. This matching is performed using string-based methods: Levenshtein distance and substring matching are used if no exact match is found. If all methods fail, no results are returned.

Each representative encodes a distinct semantic sense of the query word. For each of these semantics, the system performs a separate retrieval across the document corpus using the representative as the query vector. The results from all semantic searches are then merged and ranked to produce the final output.

To construct the final result set of size k , the following procedure is used:

1. Retrieve $\lceil k/N \rceil$ results per semantic representative, where N is the number of clusters for the matched query word.
2. Rank the aggregated results globally based on similarity to the query.

3. Select the top k results as the final output.

This strategy ensures balanced retrieval across semantic clusters and was designed to integrate with a frontend originally built for a dynamic retrieval system. It emphasizes precision but may limit semantic diversity in the top results.

3.4 Optimizations

Several optimizations were introduced to improve runtime efficiency and clustering performance:

- **PCA dimensionality reduction:** Applied prior to HDBSCAN to reduce the dimensionality of SBERT embeddings and lower computational cost.
- **Stopword removal:** Eliminated common, semantically weak tokens to improve cluster quality.
- **Vector normalization:** Embeddings were normalized to unit length, allowing cosine similarity to be computed via dot product.

3.5 Limitations

Despite its advantages, the current implementation has several limitations:

- **HDBSCAN scalability:** Iterative parameter tuning in HDBSCAN required excessive computational resources and could not be executed on a single HPC node. A workaround with using subsampling, PCA and Euclidean distance on normalized vectors was necessary. Additionally, the entire loading phase was run on a data subset. These factors may have contributed to the disparity in retrieval performance compared to static approach with k-means, as discussed in the evaluation.
- **Runtime cost:** Although clustering is performed offline, the system must still execute N full searches per query word (once per representative), which can be computationally expensive.
- **Cluster starvation:** Distributing the retrieval budget evenly across all semantic clusters does not guarantee that each cluster is included in the result set. For example, if a query word has 10 semantic clusters and $k = 11$ documents are to be returned, the system retrieves $\lceil 11/10 \rceil = 2$ documents per

cluster (20 total), and prunes to the top 11. Up to 4 clusters could be entirely excluded. This effect diminishes as k increases. For instance, at $k = 111$, each cluster yields 12 documents, and only 9 are pruned — fewer than a single cluster’s contribution.

- **String-based query-to-semantics matching:** Mapping the query to its semantic clusters is currently based on string similarity (e.g., exact matching, Levenshtein distance or substring matching). This was a design decision motivated by the fact that there is category for each token of training set (Wikipedia), making exact matching likely. However, this approach still lacks robustness, especially if the collection used for the loading phase is not the same as in the searching phase.

3.6 Reflection and Future Work

Earlier adoption of modular interfaces — such as the agreed-upon Pandas DataFrame schema — would have reduced the need for refactoring and improved system flexibility.

Future improvements include:

- Investigating non-iterative parameter tuning methods for HDBSCAN, e.g., setting fixed parameters per word class in advance.
- Replacing the inverted index with a TF-IDF-based filtering step to prioritize higher-quality documents, improve representative quality, and lower computational cost.
- Leveraging GPU-accelerated clustering and vector search libraries for scalability.
- Incorporating embedding-based matching between queries and semantic clusters to improve robustness to surface form variation. This would better capture semantic similarity but increase memory usage due to the need for additional embeddings per cluster.
- Exploring a result composition strategy prioritizing semantic diversity in the result set, similar to the one used in the dynamic approach.

4 Dynamic Approach

4.1 Overview of the Dynamic Approach

The dynamic approach takes a fundamentally different strategy compared to the static method by

performing semantic clustering at query time rather than during preprocessing. This approach first retrieves a large set of candidate documents using similarity search, then applies clustering algorithms to these candidates to ensure semantic diversity in the final results. The system leverages SBERT for sentence encoding and employs cosine similarity for initial retrieval, but distinguishes itself through runtime clustering that adapts to each specific query.

4.2 Loading Phase

The dynamic approach employs a streamlined loading phase focused on preparation rather than semantic preprocessing. During initialization, the system loads precomputed SBERT embeddings and associated text documents from compressed storage formats. A key optimization implemented is a caching mechanism that converts embeddings into memory-efficient numpy arrays.

Unlike the static approach, no clustering or semantic preprocessing occurs during this phase. Instead, the system focuses on:

- **Memory optimization:** Converting embeddings to float32 arrays for reduced memory footprint
- **Caching:** Creating preprocessed numpy ([Harris et al., 2020](#)) arrays to accelerate subsequent similarity computations

4.3 Searching Phase

The dynamic approach’s searching phase consists of two distinct stages: candidate retrieval and dynamic clustering.

Candidate Retrieval

Upon receiving the query, the system encodes it using the same SBERT model used for document embeddings. The encoded query vector then undergoes similarity search against the entire corpus using cosine similarity (implemented as dot product on normalized vectors).

To handle large-scale document collections efficiently, the system implements both CPU and GPU-accelerated search strategies:

- **GPU acceleration:** Utilizes PyTorch ([Paszke et al., 2019](#)) tensors and CUDA ([NVIDIA et al., 2020](#)) operations for batch processing
- **CPU search:** Fallback if no GPUs are available

- **Adaptive batch sizing:** Dynamically adjusts batch sizes based on available memory to prevent out-of-memory errors

The system retrieves a configurable number of top candidates (typically 1000 documents) based on similarity scores, forming the candidate pool for subsequent clustering.

Dynamic Clustering

The retrieved candidates undergo clustering using either K-means or HDBSCAN algorithms, with the choice depending on the desired clustering behavior:

- **K-means:** Provides deterministic cluster assignment with a predefined number of clusters
- **HDBSCAN:** Offers density-based clustering that can automatically determine cluster count and identify noise points

For each identified cluster, the documents are internally ranked by their original score to the original query and the system selects the document with the highest similarity score as the cluster representative.

4.4 Result Composition

The final result ranking prioritizes semantic diversity while maintaining relevance:

1. Some noise points from HDBSCAN (if applicable) are included as individual results
2. Clusters are ranked by their representative
3. Results are selected in a Round-Robin manner, so all clusters are equally represented and no cluster starves
4. Results are limited to the requested a number k

4.5 Optimizations

Several optimizations were implemented to enhance both CPU and GPU implementations:

- **GPU/CPU hybrid processing:** Automatic detection of GPU availability with fallback to CPU implementations
- **Memory-efficient similarity search:** Chunked processing prevents memory overflow

- **CuML integration:** Leverages GPU-accelerated clustering algorithms when RAPIDS CuML (Raschka et al., 2020) is available, significantly reducing clustering time

4.6 Limitations

Despite its advantages in providing semantically diverse results, the dynamic approach has several notable limitations:

- **Query-time computational cost:** Clustering operations at query time introduce significant latency compared to simple similarity ranking. Each query requires both similarity search and clustering computation, making it slower than precomputed approaches.
- **Scalability constraints:** While the chunked similarity search handles large document collections, the clustering phase becomes computationally expensive as the candidate set size increases. The current implementation limits clustering to the top 1000 candidates to maintain reasonable response times.
- **Hardware dependency:** Optimal performance requires GPU acceleration for both similarity search and clustering. On CPU-only systems, response times can become prohibitively slow for interactive applications.
- **Cluster quality variability:** The quality of semantic clustering depends on the retrieved candidate set. If initial similarity search fails to capture diverse semantic aspects, subsequent clustering cannot recover this diversity.
- **Memory requirements:** Loading entire embedding collections into memory limits the maximum corpus size that can be efficiently processed, particularly on systems with limited RAM.
- **Parameter sensitivity:** K-means and HDBSCAN clustering require careful parameter tuning that may need adjustment for different domains or query types.

4.7 Reflection and Future Work

The dynamic approach shows that query-time semantic clustering is technically feasible, though with significant practical limitations. While the

modular design made it straightforward to test different clustering algorithms, the approach revealed fundamental challenges in balancing result diversity with acceptable response times. The most significant issue encountered was the computational cost of runtime clustering. Even with GPU acceleration, query response times often exceeded what would be acceptable for interactive applications. Additionally, the quality of clustering heavily depends on the initial candidate retrieval - if the similarity search misses important semantic aspects, clustering cannot recover them. The implementation also highlighted infrastructure dependencies that limit practical deployment. Requiring GPU acceleration for reasonable performance and needing substantial memory for large document collections creates barriers for broader adoption.

Future improvements should focus on:

- **Hybrid approaches:** Combining benefits of static preprocessing with dynamic adaptation, such as pre-clustering documents into coarse semantic groups and performing fine-grained clustering at query time
- **Multi-stage retrieval:** Developing hierarchical search strategies that first identify semantic regions, then perform detailed clustering within relevant regions

5 Queries and Evaluation

5.1 Evaluation Metrics

To evaluate our IR approaches, we used precision, recall, and F1 score to measure retrieval accuracy. We also included heterogeneity and F1 heterogeneity to assess how well the results reflected the range of possible meanings.

5.2 Initial usage of Wikidata

To perform the evaluation, we first required a set of queries and a consistent method for assessing their results. For this purpose, we selected a set of single-word homonyms as queries, as these naturally involve lexical ambiguity and present a suitable challenge for IR models. We determined that each query should be linked to clearly defined senses in order to support a fair and reproducible evaluation. To establish these sense mappings, we used Wikipedia disambiguation pages, which group together distinct meanings associated with ambiguous terms. To extract and structure this data, we relied on Wikidata ([Vrandečić and Krötzsch, 2014](#)),

a curated knowledge base that connects entities and Wikipedia articles through unique identifiers. Our goal was to utilize a tool that could support fast searching, semantic representation, and direct one-to-one mapping between homonyms and their corresponding Wikipedia entries. Wikidata enabled this through its underlying data model and SPARQL query interface ([Harris and Seaborne, 2013](#)). Using this setup, we retrieved a list of single-word homonyms and mapped each to its possible senses, represented by distinct Wikipedia pages. These mappings allowed us to build an indexed ground truth dataset in which each homonym was associated with a set of documents. This dataset served as the foundation for systematically evaluating model performance under lexical ambiguity.

Results

Although the setup offered an interpretable mapping between homonyms and their senses, its performance fell short of expectations in practice. Figure 2 shows the results for the dynamic approach using HDBSCAN clustering, which are representative of the overall poor performance observed across different methods, including simple SBERT retrieval without clustering. Precision, recall and F1-measure are all quite low across the evaluated queries.

The results reveal several concerning patterns. Precision values cluster around 0.02-0.04, indicating that the vast majority of retrieved documents were not relevant according to the Wikidata ground truth mappings. Recall shows slightly better performance with a median around 0.15-0.20, but with high variability across different queries, suggesting inconsistent coverage of the relevant document sets. The F1 scores, combining both precision and recall, remain disappointingly low at approximately 0.02-0.05 for most queries.

The heterogeneity scores present a mixed picture, with values ranging from 0.4 to 1.0 and numerous outliers at the maximum value. While high heterogeneity might initially seem positive for semantic diversity, the extremely low precision and recall scores suggest that this diversity comes at the cost of relevance - the system appears to be retrieving semantically varied but largely irrelevant documents.

The consistently low performance observed across different methods led us to reconsider our evaluation methodology and explore alternative approaches for assessing semantic retrieval quality.

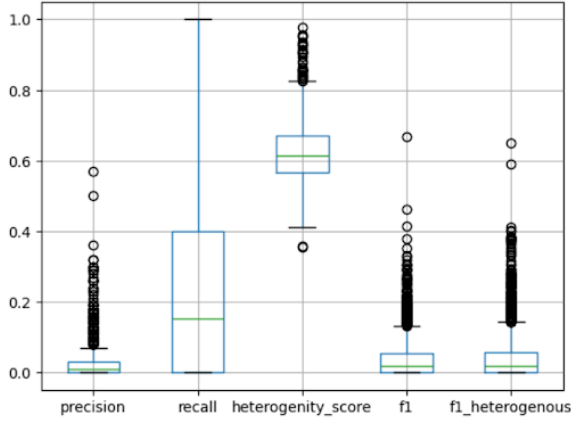


Figure 2: Results for the first evaluation Method, using the dynamic approach with HDBSCAN

5.3 Transformer-Based Evaluation

Because our first evaluation approach of using Wikipedia’s disambiguation pages in combination with Wikidata did not work as expected, we decided to use a different evaluation method. As the main problem of the first approach was that the documents collected from the disambiguation pages didn’t contain all relevant documents belonging to a search query and therefore always resulted in low precision and recall, this was the main problem we wanted to solve with the second evaluation approach.

To address this, we developed a transformer-based evaluation strategy that replaces the disambiguation-page-based document sets with a more reliable reference: the results retrieved by ColBERT, a state-of-the-art information retrieval system. In particular, we took the top-100 results from the ColBERT search as ground truth for relevant documents for a query. Even though this approach has the downside that it is based on BERT (Devlin et al., 2019) and therefore might be favorably inclined towards approaches that use BERT as part of their information retrieval system as well, we believe it can still offer valuable insights into the different approaches. This is because the standard SBERT as well as our two approaches are all BERT-based as well and can therefore be evaluated comparatively against each other.

Results

Although the results of the second evaluation method, seen in Figure 3, look a bit more promising than the results of the first evaluation method, especially with regards to precision, it still falls short of our expectations. This is because the values for

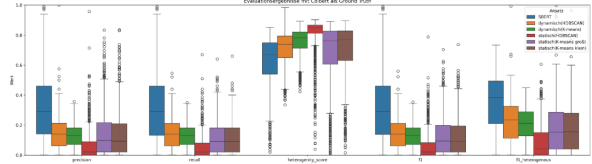


Figure 3: Results of the second evaluation method.

precision and recall are still low.

This means that because the heterogeneity of the search results is relatively high for all methods, the F1 heterogeneity scores mainly depend on the precision score. This defeats the purpose of the F1 heterogeneity score to a large extent, because it was meant to balance high heterogeneity with high precision. Nonetheless, some interesting findings present themselves from the evaluation. Firstly, unmodified BERT-based information retrieval methods are already successful in achieving heterogeneous search results. This can be seen by the high heterogeneity score of SBERT. Secondly, our approaches achieved their goal of modifying the information retrieval process in a way that increases the heterogeneity of the search results. This can be seen by the comparatively higher heterogeneity scores of our approaches.

Acknowledgments

We would like to thank our tutor Julius Gonsior for his guidance and support throughout this project.

6 Conclusion

In this project, we set out to design and evaluate a static and a dynamic IR system. The goal of both systems was to improve the diversity of search results for homonym queries. Wikipedia and Wikidata were used as evaluation datasets to explore how retrieval methods can be adapted to better handle semantic ambiguity.

The static approach learns the semantics of words during a training phase. At query time, it uses precomputed semantic representations to rank documents, which removes the need for further processing. This method is intended to ensure full semantic coverage, as long as the relevant meanings are included during the loading phase. It uses SBERT for sentence encoding and cosine similarity for ranking. The dynamic approach takes a different direction by performing semantic clustering at query time. It first retrieves a large set of candidate documents using similarity search, then

applies clustering to ensure that the final results represent different meanings. Like the static method, it uses SBERT and cosine similarity, but adapts its behavior to each specific query through runtime clustering.

For the first evaluation, we selected single-word homonyms and linked them to their possible meanings using Wikipedia disambiguation pages. Wikidata helped structure these mappings by connecting each homonym to specific Wikipedia pages. This allowed us to create a dataset where each query was linked to a defined set of documents. Although this approach provided a clear evaluation framework, it did not perform well in practice. Many relevant documents were missing, which led to low precision and recall. To improve this, we introduced a second evaluation based on ColBERT, a transformer-based IR system. We used its top 100 results for each query as a new reference set. While this approach may introduce some bias in favor of BERT-based systems, it still allowed for a more consistent comparison, since our methods and SBERT are also based on BERT. The second evaluation showed slightly better results, especially in terms of precision. However, both systems still fell short of our expectations. Precision and recall remained low, which reduced the value of the F1 heterogeneity score. Since heterogeneity was high across all systems, the F1 score depended mostly on precision, which limited its usefulness as a balanced measure of diversity and accuracy.

Despite these challenges, the evaluation led to several valuable insights. First, standard BERT-based retrieval methods like SBERT already achieve a high level of heterogeneity. Second, our methods were able to guide the retrieval process toward more diverse results, as shown by their higher heterogeneity scores compared to the baseline. These results suggest that while improving both precision and diversity at the same time remains difficult, it is possible to influence the retrieval process in a way that produces more varied results. Future work should focus on better combining these two aspects so that evaluation metrics like the F1 heterogeneity score can more accurately reflect the overall performance of IR systems under lexical ambiguity.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep](#)

[bidirectional transformers for language understanding](#). *Preprint*, arXiv:1810.04805.

F.B. Dian Paskalis and M.L. Khodra. 2011. [Word sense disambiguation in information retrieval using query expansion](#). In *Proceedings of the 2011 International Conference on Electrical Engineering and Informatics*, pages 1–6.

Marcos García. 2021. [Exploring the representation of word meanings in context: A case study on homonymy and synonymy](#). *CoRR*, abs/2106.13553.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, and 7 others. 2020. [Array programming with NumPy](#). *Nature*, 585(7825):357–362.

Steve Harris and Andy Seaborne. 2013. SPARQL 1.1 query language. <https://www.w3.org/TR/sparql11-query/>. W3C Recommendation.

Omar Khattab and Matei Zaharia. 2020. [Colbert: Efficient and effective passage search via contextualized late interaction over bert](#). *Preprint*, arXiv:2004.12832.

Robert Krovetz and W. Bruce Croft. 1992. [Lexical ambiguity and information retrieval](#). *ACM Trans. Inf. Syst.*, 10(2):115–141.

Rowan Hall Maudslay and Simone Teufel. 2022. [Homonymy information for english wordnet](#). *Preprint*, arXiv:2212.08388.

Leland McInnes, John Healy, and Steve Astels. 2017. [hdbscan: Hierarchical density based clustering](#). *Journal of Open Source Software*, 2(11):205.

NVIDIA, Péter Vingelmann, and Frank H.P. Fitzek. 2020. [Cuda, release: 10.2.89](#).

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. [Pytorch: An imperative style, high-performance deep learning library](#). *Preprint*, arXiv:1912.01703.

Sebastian Raschka, Joshua Patterson, and Corey Nolet. 2020. Machine learning in python: Main developments and technology trends in data science, machine learning, and artificial intelligence. *arXiv preprint arXiv:2002.04803*.

Nils Reimers and Iryna Gurevych. 2019. [Sentence-bert: Sentence embeddings using siamese bert-networks](#). In *Proceedings of the 2019 Conference on Empirical*

Methods in Natural Language Processing. Association for Computational Linguistics.

Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A free collaborative knowledge base. *Communications of the ACM*, 57(10):78–85.

Junjie Wu. 2012. *Advances in K-means clustering: a data mining thinking*. Springer Science & Business Media.

A Search Interface

To aid evaluation and comparison of the different approaches, we developed a terminal based interface with textual². It allows interactive testing of the static, dynamic, and baseline (no-cluster) methods. The interface provides a user-friendly way to explore how each approach handles semantic ambiguity and result diversity.

A.1 Interface Features

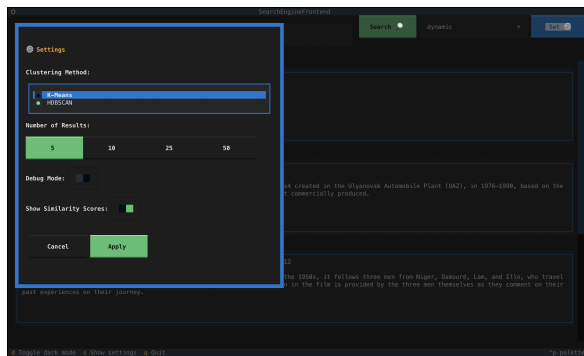


Figure 4: Search interface settings dialog.

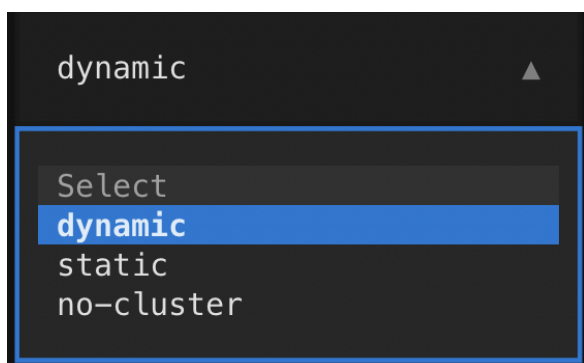


Figure 5: Approach selection dropdown.

The search interface includes several configurable options:

- **Approach Selection:** Users can switch between the static approach, dynamic approach,

and a baseline without clustering to directly compare their behavior on identical queries.

- **Clustering Method:** For the dynamic approach, users can choose between K-means and HDBSCAN clustering algorithms to observe their different clustering behaviors.
- **Result Count:** Configurable number of results (5, 10, 25, or 50) to examine how each approach performs with different result set sizes.
- **Similarity Scores:** Optional display of similarity scores to provide insight into the ranking decisions made by each approach.
- **Cluster Information:** Results include cluster assignments and original rankings, allowing users to see how clustering affects the final result composition.

A.2 Example Query Results

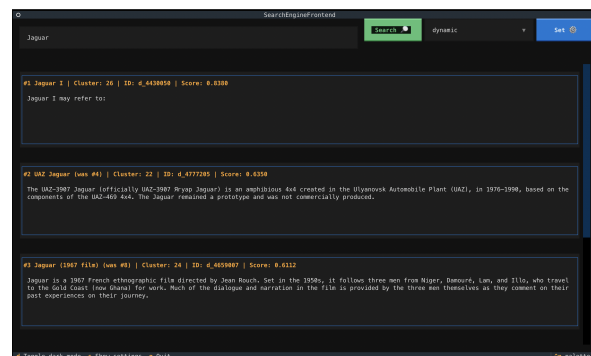


Figure 6: Search results for "Jaguar" showing semantic diversity.

The interface effectively demonstrates the semantic diversity achieved by the approaches. For instance, a search for "Jaguar" successfully retrieves documents covering different semantic senses: an animal reference, the UAZ Jaguar vehicle, and the 1967 French film "Jaguar." Each result displays its cluster assignment, original ranking, and similarity score.

²<https://textual.textualize.io>